AI & DATA SCIENCE TRACK · ROUND 2 · NHA-4-190

# AI-Powered Predictive Maintenance for Industrial Equipment

An end-to-end machine learning system that uses IoT sensor data — temperature, torque, rotational speed, tool wear — to predict equipment failures before they occur, enabling just-in-time maintenance at industrial scale.

**10,000**
Sensor observations analyzed

**4**
ML models trained and evaluated

**0.48**
Optimal $F_2$-tuned threshold

**5**
REST API endpoints deployed

TABLE OF CONTENTS

SECTION 01

## Problem Statement

Industrial equipment failures cause severe operational disruption and financial loss. The status quo — reactive or fixed-schedule maintenance — is both costly and inefficient.

| $50B | 30–50% | 70% |
| --- | --- | --- |
| Annual unplanned downtime cost worldwide | Of all maintenance is still reactive | Of failures preventable with early detection |

- **Reactive maintenance** (fix-when-broken) leads to cascading equipment damage, safety hazards, production halts, and high emergency repair costs.
- **Scheduled maintenance** (time-based) wastes labor and spare parts servicing equipment that doesn't yet need it.
- **Predictive maintenance** uses real-time sensor data and ML to identify degradation before failure — enabling intervention at exactly the right time.

---

SECTION 02

## Dataset

The project uses the **AI4I 2020 Predictive Maintenance Dataset** from the UCI Machine Learning Repository — a synthetic dataset reflecting real-world industrial equipment behavior.

| 10,000 | 14 | 3.4% |
| --- | --- | --- |
| Total observations | Raw features | Positive class (failure) rate |

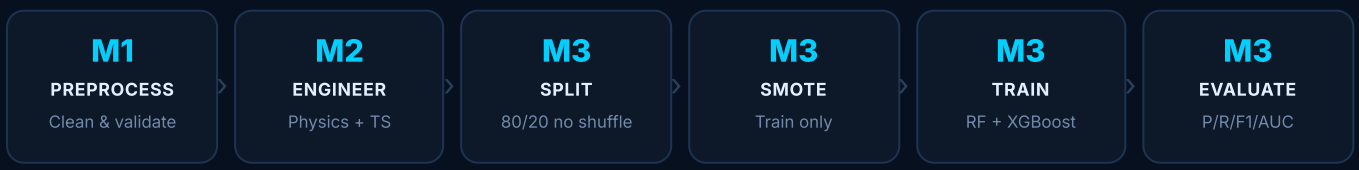
FEATURE REFERENCE

| FEATURE | UNIT | DESCRIPTION | TYPE |
|---|---|---|---|
| Air Temperature | Kelvin | Ambient air temperature around the equipment | `Numeric` |
| Process Temperature | Kelvin | Internal operating temperature | `Numeric` |
| Rotational Speed | RPM | Spindle rotation speed | `Numeric` |
| Torque | Nm | Torque applied during operation | `Numeric` |
| Tool Wear | Minutes | Cumulative tool usage time | `Numeric` |
| Type | L / M / H | Product quality type — Low / Medium / High | `Categorical` |
| TWF, HDF, PWF, OSF, RNF | Binary | Individual failure mode indicators | `Binary` |
| **Machine Failure** | Binary | **Target variable — 1 if machine failed** | `Target` |

- **Severe class imbalance:** Only ~3.4% of samples represent failures — requiring oversampling.
- **No missing values:** Zero null entries across all columns.
- **Temporal ordering:** Rows are ordered sequentially, simulating live sensor readings. Shuffle must be disabled in the split.

SECTION 03

## Methodology




### 4.1 — DATA PREPROCESSING (MILESTONE 1)

Implemented in `src/data_preprocessing.py`. Non-predictive identifiers ( `UDI` , `Product ID` ) were dropped and column names cleaned of unit annotations (e.g. `Air temperature [K]` → `Air_Temp` ). Zero null values confirmed. All configuration centralized in `config/config.yaml` — nothing hardcoded.

### 4.2 — FEATURE ENGINEERING (MILESTONE 2)

#### PHYSICS-INFORMED FEATURES

| FEATURE | FORMULA | RATIONAL |
|---|---|---|
| `Temp_Diff` | Process_Temp − Air_Temp | Thermal stress indicator |
| `Power` | Torque × Rotational_Speed | Mechanic load prox |

#### TIME-SERIES FEATURES (WINDOW = 5)

| FEATURE | RATIONALE |
|---|---|
| `Temp_Rate_of_Change` | Rapid thermal shifts |
| `Torque_Rate_of_Change` | Mechanical stress spikes |
| `Rolling_Avg_Temp` | Smoothed temp trend |
| `Rolling_Std_Torque` | Torque instability |

The `Type` column (L/M/H) was one-hot encoded with `drop_first=True`. **PCA was excluded from the production pipeline** — used for visualization only in the notebook.

### 4.3 — DATA SPLITTING & BALANCING

#### TRAIN / TEST SPLIT — 80/20

`shuffle=False` preserves temporal ordering — critical for time-series integrity. Random shuffling would leak future data into the training window.

#### SMOTE OVERSAMPLING

Applied **only to the training set** to address the 3.4% failure rate. Never applied to the test set — preventing data leakage.

### 4.4 — MODEL DEVELOPMENT (MILESTONE 3)

| MODEL | CONFIGURATION | OPTIMIZATION | $F_2$ THRESHOLD |
|---|---|---|---|
| Random Forest — Baseline | `n_estimators=100`, default | None | `0.45` |
| **Random Forest — Optimized** | GridSearch: trees, depth, splits | 3-fold CV, recall scoring | `0.48` |
| XGBoost — Baseline | `eval_metric=logloss` | None | `0.81` |
| XGBoost — Optimized | GridSearch: depth, LR, n_est | 3-fold CV, recall scoring | `0.47` |

### 4.5 — THRESHOLD OPTIMIZATION

Standard 0.5 thresholds are suboptimal — the cost of a missed failure far exceeds a false alarm. Thresholds were optimized using **F-beta score with β=2**, weighting recall twice as heavily as precision. Thresholds are serialized to JSON in `/models` and loaded at API startup.

### 4.6 — EVALUATION METRICS

**PRECISION**

Fraction of predicted failures that were actual failures

**RECALL**

Fraction of actual failures correctly predicted — primary metric

**F1-SCORE**

Harmonic mean of precision and recall

**ROC-AUC**

Discriminative ability across all thresholds

SECTION 04

## Deployment Architecture

### 5.1 — FASTAPI REAL-TIME SERVICE

| | | |
|---|---|---|
| GET | `/health` | Liveness check — returns model name, threshold, expected feature count |
| POST | `/predict` | Accept sensor payload → compute features → return failure probability + binary decision |
| POST | `/feedback` | Submit true outcome labels tied to a request_id for online evaluation |
| GET | `/metrics` | Prediction volume and alert rate from the prediction log |
| GET | `/drift` | PSI drift check — avg PSI, per-feature scores, drift detected flag |

### 5.2 — ONLINE FEATURE GENERATION

The API mirrors the training feature pipeline in real time via `src/serving.py`. The `RollingFeatureStore` maintains a per-asset-ID rolling window deque, computing time-series features identically to the offline pipeline. Features are aligned to `model.feature_names_in_` to guarantee column ordering before inference.

### 5.3 — CONTINUOUS MONITORING

**DATA DRIFT — PSI**

Population Stability Index between training and live distributions, per feature. **Alert: PSI > 0.20**

**ONLINE PERFORMANCE**

Precision, Recall, F1, ROC-AUC from feedback labels on live samples. **Recall threshold: 0.80**

**AUTO-RETRAINING**

When thresholds breached, `monitor.py --retrain` triggers full pipeline re-execution automatically.

### 5.4 — VERSIONING & REPRODUCIBILITY

- All hyperparameters and paths centralized in `config/config.yaml` — nothing hardcoded.
- Model artifacts serialized with **joblib** to `/models`. Threshold metadata stored as JSON alongside each model.
- Full pipeline reproducible with: `python main.py` — deterministic seeds throughout.

SECTION 05

## Results & Key Findings

### 6.1 — MODEL PERFORMANCE SUMMARY

| MODEL | $F_2$ THRESHOLD | NOTES |
|---|---|---|
| Random Forest — Baseline | `0.45` | Solid baseline with good recall |
| **Random Forest — Optimized** | `0.48` | **Selected for deployment** — best recall + generalization |
| XGBoost — Baseline | `0.81` | High threshold — conservative predictions |
| XGBoost — Optimized | `0.47` | Competitive after tuning |

### 6.2 — FEATURE IMPORTANCE (RANDOM FOREST)


### 6.3 — DATA INSIGHTS

- The 3.4% failure rate confirms severe class imbalance, validating the use of SMOTE.

- Degradation patterns visible: temperature trends upward and torque variance increases in the **20–50 steps preceding failure events**.
- PCA reveals **partial but real separation** between normal and failure clusters — engineered features carry discriminative signal.
- Binary failure-type columns (TWF, HDF, PWF, OSF) are highly predictive when active, but sparse in practice.

---

SECTION 06

## Challenges & Solutions

| | |
|---|---|
| Severe class imbalance — only 3.4% failures | SMOTE applied to training data only + $F_2$ threshold optimization (β=2 penalizes missed failures 2× more than false alarms) |
| Temporal data leakage risk | `shuffle=False` in train/test split preserves time ordering; SMOTE applied only after splitting to prevent test set contamination |
| Online feature parity — matching training pipeline in production | `RollingFeatureStore` in `serving.py` replicates rolling-window logic per asset ID, perfectly mirroring offline feature engineering |
| Model drift in production | PSI-based monitoring detects distribution shift; automated retraining re-executes when PSI > 0.20 or recall < 0.80 |
| Threshold selection — 0.5 default is wrong for this task | F-beta (β=2) optimization on precision-recall curves; thresholds saved as JSON and loaded by API at startup |
| Reproducibility across environments | Central `config/config.yaml` as single source of truth; deterministic random seeds; single-command pipeline |

---

SECTION 07

## Business Impact

**30–50%**

Reduction in unplanned downtime (McKinsey, 2020)

**20–40%**

Extension of equipment lifespan

1. **Reduced unplanned downtime:** Failures predicted before occurrence — maintenance scheduled during planned windows, avoiding production halts.
2. **Cost savings:** Eliminates emergency repair costs, reduces spare parts inventory, cuts overtime labor from reactive crises.
3. **Extended equipment lifespan:** Early detection of degradation enables interventions that prevent cascading damage.
4. **Optimized scheduling:** Condition-based maintenance replaces fixed-interval servicing.
5. **Operational visibility:** Real-time API provides continuous, per-asset health monitoring across entire industrial fleets.

---

SECTION 08

## Future Work

**LSTM / TRANSFORMERS**

Deep learning for sequential sensor data — longer temporal dependencies beyond 5-step rolling windows

**MULTI-CLASS FAILURE PREDICTION**

Predict specific failure type — TWF, HDF, PWF, OSF, RNF — enabling targeted maintenance actions

**REMAINING USEFUL LIFE (RUL)**

Predict not just *if* but *when* — precision maintenance scheduling with time horizons

**EDGE DEPLOYMENT**

Compile model for IoT edge devices co-located with equipment — eliminating network latency

**WEB DASHBOARD**

Real-time fleet health dashboard for maintenance teams — alerts, trends, historical analytics

**A/B TESTING FRAMEWORK**

Controlled production experiments to validate retraining improves real-world recall

---

SECTION 09

## Project Structure

```
NHA-4-190/ ├── config/config.yaml # All hyperparameters and paths ├── data/raw/ # Place ai4i2020.csv here ├── models/ # .joblib models + threshold JSON files ├── monitoring/monitor.py # PSI drift, online metrics, auto-retraining ├── notebooks/visualization.ipynb # EDA, PCA, confusion matrices ├── outputs/plots/ # Confusion matrix PNGs ├── outputs/monitoring/ # predictions_log.csv, feedback_log.csv, report ├── src/data_preprocessing.py # M1: Load, clean, validate ├── src/feature_engineering.py # M2: Physics + time-series + categorical ├── src/model_training.py # M3: Split, SMOTE, RF, XGBoost, GridSearch ├── src/model_evaluation.py # M3: Metrics, confusion matrices, thresholds ├── src/serving.py # M4: Online feature store, PSI, CSV logging ├── app.py # M4: FastAPI deployment service └── main.py # Pipeline entry point (M1-M3)
```

SECTION 10

## How to Reproduce

```
# 1. Install dependencies pip install -r requirements.txt # 2. Place dataset cp /path/to/ai4i2020.csv data/raw/ # 3. Run full training pipeline (Milestones 1-3) python main.py # 4. Start real-time API (Milestone 4) uvicorn app:app --host 0.0.0.0 --port 8000 # 5. Run monitoring check python monitoring/monitor.py # 6. Auto-retrain when thresholds breached python monitoring/monitor.py --retrain --skip-optimization
```

SECTION 11

## Conclusion

This project demonstrates a complete, production-ready predictive maintenance pipeline — from raw IoT sensor data to a deployed real-time inference API with continuous monitoring and automated retraining. The system addresses class imbalance, temporal data integrity, and model drift, producing a robust solution that can meaningfully reduce industrial downtime and maintenance costs.

The **optimized Random Forest model**, combined with F-beta threshold tuning (β=2) and PSI-based drift detection, provides a strong and interpretable foundation for predictive maintenance. The modular architecture and centralized configuration make it straightforward to extend, retrain, and adapt to new equipment types.

| 5 | 4 | 5 | PSI |
|---|---|---|---|
| Milestones delivered | ML models trained | API endpoints live | Automated drift monitoring |